TinyNav: End-to-End Image-Goal Navigation on a Microcontroller

Introduction

Image-goal navigation (ImageNav) is the task of directing a mobile agent to a target location specified by a photograph of that location. Existing ImageNav methods follow either modular or end-to-end paradigms. Modular pipelines decompose the problem into separate components for mapping, localization, and planning, each independently designed and maintained [2, 6]. End-to-end approaches collapse this pipeline into a single learned model that maps raw observations directly to actions, offering greater efficiency and eliminating error propagation between modules [24]. Unlike point-goal navigation, which assumes access to GPS or metric pose information, ImageNav requires the agent to visually recognize the spatial relationship between its current observation and the goal, and to select actions that reduce this discrepancy over time. This eliminates the need for pre-built maps, localization infrastructure, or environment-specific calibration, making it applicable to building navigation robots using only visual inputs. However, it introduces the complexity of requiring a single learned policy to implicitly solve visual place recognition, relative pose estimation, and obstacle-aware path planning.

Recent work has produced increasingly capable ImageNav systems. Large-scale navigation foundation models such as GNM [17], ViNT [18], and NoMaD [19] demonstrate zero-shot transfer across robot platforms through pre-training on massive multi-robot datasets, but adapting them to a specific environment remains computationally expensive and often underperforms purpose-built policies [11, 3]. Monaci et al. [13] show that architectural choices in the perception module, particularly early fusion between goal and observation, matter more than backbone scale for downstream navigation performance. MINav [11] takes the opposite approach to foundation models, combining unsupervised pink-noise exploration with hindsight goal relabeling and offline reinforcement learning to train navigation policies from purely autonomous experience in under two hours, without human demonstrations or reward engineering. RSRNav [15] introduces cross-correlation mechanisms that compute dense spatial correspondences between goal and observation feature maps, producing compact cue vectors that encode directional information rather than raw pixel similarity. Together, these advances represent a shift toward systems that can learn to navigate from self-supervised interaction and reason about space at the level of feature geometry rather than appearance.

However, deploying these systems on physical robots remains constrained by hardware cost and sensor assumptions. Existing edge deployments typically rely on NVIDIA Jetson platforms [17, 19], which cost over a hundred dollars, consume several watts, and impose form factor constraints unsuitable for small battery-powered robots. Grover et al. [5] identify compute latency, memory bandwidth, and power constraints as coupled barriers that dictate deployment reliability for embodied foundation models, arguing that reliable deployment requires system-level co-design rather than model compression alone. Even recent safety-aware systems such as SIGN [23], which deploys ImageNav on drones with depth-based obstacle avoidance, still assume GPU-class hardware for policy inference. Many approaches also depend on LiDAR for obstacle

avoidance, adding significant cost and mechanical complexity. Data efficiency presents a further bottleneck, as most methods require millions of frames collected in simulation [16] or through extensive teleoperation. MINav addresses the data problem through unsupervised exploration, but its original formulation still assumes LiDAR-derived depth for obstacle avoidance during data collection.

This project addresses both constraints through a pipeline that deploys an end-to-end ImageNav system on an ESP32-S3 microcontroller costing under ten dollars. A teacher model (21M parameters) is trained via IQL on autonomously collected data, using pink-noise exploration with infrared proximity-based obstacle avoidance instead of LiDAR. A student model (1M parameters) replaces standard convolutions with depthwise-separable blocks and the learned correlation with a parameterless dot-product scheme. The student is then compressed through dual-level knowledge distillation and INT8 quantization-aware training into TensorFlow Lite Micro models occupying roughly one megabyte of flash, running at 5 Hz entirely on-device with no cloud connectivity.

Background

Image-Goal Navigation

Image-goal navigation requires an agent to navigate to a target location specified by a photograph, using only egocentric visual observations and a discrete action space [24]. Unlike point-goal navigation, which provides metric displacement vectors [1], image-goal navigation demands that the policy implicitly perform visual place recognition and relative spatial reasoning from pixels alone. Several approaches have been proposed to improve the perceptual component of navigation policies. ZSON [12] leverages CLIP embeddings to improve zero-shot generalization across environments, while FGPrompt [20] uses fine-grained goal prompting to extract observation features conditioned on the goal image. Monaci et al. [13] provide a systematic analysis showing that architectures supporting implicit correspondence computation, particularly early patch-wise fusion and cross-attention, significantly outperform late-fusion alternatives. SIGN [23] extends ImageNav to autonomous drones with a depth-based safety module for obstacle avoidance, though it still requires GPU-class hardware for inference. The standard evaluation protocol, established by the Habitat benchmark [16], measures success rate and Success weighted by inverse Path Length (SPL) [1], rewarding policies that reach the goal efficiently.

MINav and Offline Reinforcement Learning

Training image-goal navigation policies is complicated by the absence of a natural reward signal. MINav [11] addresses this through unsupervised exploration followed by retroactive goal labeling, eliminating the need for human supervision or simulation.

The robot first explores using temporally correlated actions generated by $1/f$ (pink) noise. Unlike white noise, which produces jittery trajectories that fail to cover space, pink noise introduces temporal autocorrelation that yields smooth, sweeping motions. The noise is generated through the Voss-McCartney algorithm, which layers multiple random sources at different update frequencies to approximate a $1/f$ spectrum without requiring a Fourier transform.

The collected trajectories are then labeled using *hindsight relabeling* [**?**]. For each timestep t, a future frame x_{t+k} is designated as the goal the agent was "trying to reach," with a positive reward if k is small and a step penalty otherwise. This transforms aimless exploration data into a goal-conditioned reinforcement learning dataset without modifying the collection process.

The resulting dataset is used to train a policy via Implicit Q-Learning (IQL) [8], an offline RL algorithm that avoids querying out-of-distribution actions. Offline RL methods learn from fixed

datasets without environment interaction [**?**], but standard Q-learning algorithms suffer from catastrophic overestimation of Q-values for unobserved actions. Conservative methods such as CQL [**?**] address this by penalizing Q-values on unseen actions, while policy-constraint methods such as TD3+BC [4] regularize the learned policy to stay close to the data distribution. Recent work by Park et al. [14] demonstrates that offline goal-conditioned RL can scale to long-horizon tasks through hierarchical value decomposition, further validating offline learning as a viable paradigm for navigation. IQL takes a different approach, avoiding the overestimation issue entirely by never evaluating the Q-function on actions outside the dataset. It learns a state value function through asymmetric expectile regression,

$$\mathcal{L}_V = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\tau\left(Q_{\text{target}}(s,a) - V(s)\right)\right] \tag{1}$$

where $L_2^\tau(u) = |\tau - \mathbf{1}(u<0)| \cdot u^2$ and $\tau > 0.5$ biases the value estimate toward upper quantiles, implicitly performing policy improvement without maximizing over unseen actions. The Q-function is trained with temporal difference learning,

$$\mathcal{L}_Q = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma(1-d)V(s') - Q(s,a)\right)^2\right] \tag{2}$$

and the policy is extracted by advantage-weighted log-likelihood,

$$\mathcal{L}_\pi = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\exp(\beta\left(Q(s,a) - V(s)\right)) \cdot \log \pi(a|s)\right] \tag{3}$$

where β controls the sharpness of advantage weighting.

RSRNav and Cross-Correlation Spatial Reasoning

Comparing goal and observation images through pixel difference or feature concatenation discards spatial structure. Prior methods such as OVRL-V2 [22] improve perceptual quality through strong visual backbones pre-trained with self-supervised learning, but rely on rich semantic features without explicitly modeling spatial correspondence. RSRNav [15] instead computes dense spatial correspondences between goal and observation feature maps, drawing on the dense matching paradigm established by optical flow methods such as RAFT [21] and demonstrating that explicit correlation outperforms stronger encoders without it.

Given a shared encoder producing feature tensors $\mathbf{F}_g, \mathbf{F}_o \in \mathbb{R}^{C\times H\times W}$, the correlation module L2-normalizes each spatial position along the channel dimension and computes pairwise dot products, yielding a correlation volume $\mathbf{C} \in \mathbb{R}^{HW\times HW}$ that captures which parts of the goal correspond to which parts of the current view. This volume is refined through a correlation pyramid (progressively pooling the goal-side resolution for coarse-to-fine correspondence) and direction-aware local lookups (extracting small neighborhoods around corresponding positions). The resulting features are fused and projected to a compact cue vector that encodes *how to get from here to there* rather than what each scene contains independently.

Model Compression for Microcontrollers

Deploying neural networks on microcontrollers requires compression techniques that reduce parameters and compute by orders of magnitude.

Knowledge distillation [**?**] trains a small student to mimic a large teacher's softened output distribution. The softmax temperature T controls entropy, with higher temperatures revealing inter-class relationships that hard labels cannot capture. This is particularly effective in reinforcement learning, where the "correct" action depends on a complex value landscape.

Quantization-aware training (QAT) [**?**] inserts fake quantization nodes during training, simulating INT8 rounding in the forward pass while using straight-through gradient estimation

in the backward pass. The resulting model executes entirely in integer arithmetic, which is critical for microcontrollers lacking hardware floating-point support.

Depthwise-separable convolutions [7] factorize standard convolutions into a per-channel depthwise pass and a 1×1 pointwise mix, reducing multiply-accumulate operations by approximately 8-9$\times$ for typical configurations. MobileNetV2 [**?**] further improves this with inverted residual blocks and linear bottlenecks. Combined with distillation and quantization, depthwise-separable blocks form the standard compression stack for microcontroller vision models [9].

TensorFlow Lite Micro serves as the deployment runtime, providing a minimal interpreter that executes TFLite flatbuffer graphs using only static memory allocation from a caller-provided arena, with optimized INT8 kernels and a code footprint of approximately 20 KB. TinyEngine [10] further demonstrates that inference-time memory optimization can enable on-device training under 256 KB, underscoring the feasibility of neural network execution on severely constrained hardware. Grover et al. [5] provide a comprehensive survey of the systems-level barriers to deploying embodied foundation models at the edge, cataloguing eight coupled failure modes spanning compute, memory, and communication, and arguing that co-design across these layers is necessary for reliable real-time control.

Method

Data Collection

The robot platform consists of an ESP32-S3 MCU driving with a fisheye camera, and an Arduino that provides heading and proximity readings over UART at 50 Hz. The Arduino runs a complementary filter on gyroscope and magnetometer for heading estimation, while its onboard infrared sensor provides short-range obstacle detection on a 0–255 scale (0 indicating contact, 255 indicating clear).

During exploration, the robot executes temporally correlated actions generated by pink noise via the Voss-McCartney algorithm. A proximity-based safety override replaces any planned forward action with a random turn when the APDS9960 reading falls below 50, preventing collisions without requiring a learned model. Each step is streamed over WiFi as a binary packet containing the JPEG frame, executed action, heading, proximity, and timestep, and logged on a laptop receiver as a 96×96 image and a JSON trajectory entry.

Multiple sessions are merged and segmented, then hindsight goal relabeling is applied. For each timestep t, K future frames are sampled as synthetic goals from $\{t+1, \ldots, \min(t+H, T)\}$, producing transitions with reward $+1$ if the goal is within three steps and -0.01 otherwise. The proximity reading at each timestep is indexed alongside the transition.

Teacher Architecture

The teacher follows the encoder-correlation-policy structure of RSRNav, extended with IQL value heads, totaling approximately 21 million parameters.

A weight-shared ResNet encoder with three stages (channel widths 128, 256, 512 with 3, 4, 3 residual blocks) maps both the observation and goal from $96 \times 96 \times 3$ to $12 \times 12 \times 512$ feature maps. A direction-aware pyramid correlation module computes multi-scale cross-correlation between the two feature maps, applies local lookups, and fuses the result through convolutional layers into a 512-dimensional cue vector. The policy network concatenates this cue with a normalized proximity scalar and maps the 513-dimensional input to four action logits through a three-layer MLP (256, 128 hidden units). Twin Q-networks and a V-network, conditioned on the same cue and proximity, provide the value estimates required by IQL.

The teacher is trained with the IQL losses in Equations (1)–(3) using Adam (lr $= 3 \times 10^{-4}$, $\tau = 0.7$) with Polyak-averaged target Q-networks.

Student Architecture

The student replaces every component of the teacher with a hardware-efficient alternative while preserving the encoder-correlation-policy structure, totaling approximately one million parameters.

The encoder uses a four-stage hierarchy of depthwise-separable convolutions. A stride-2 stem reduces the input to 48×48 at 128 channels, and three subsequent stages (widths 256, 512, 1024 with 3, 4, 3 blocks each) progressively downsample to a $6 \times 6 \times 1024$ feature tensor. The teacher's learned correlation module is replaced by a parameterless operation that adaptively average-pools both feature maps to a 4×4 grid, L2-normalizes each of the 16 spatial positions, and computes the flattened $16 \times 16 = 256$ matrix of pairwise dot products. Two additional scalars capture left-right directional bias through column-wise cosine similarity, yielding a 258-dimensional cue at zero parameter cost. The proximity reading, normalized by dividing by 255, is concatenated to form a 259-dimensional input to a four-layer MLP (hidden widths 512, 256, 128) that produces the four action logits.

Model Compression Pipeline

Knowledge Distillation

The student is trained to match the teacher at two levels. At the action level, the student minimizes KL divergence against the teacher's tempered output distribution,

$$\mathcal{L}_{\text{action}} = T^2 \cdot D_{\text{KL}}\big(\sigma(z^S/T) \parallel \sigma(z^T/T)\big) \tag{4}$$

with temperature $T = 4$, supplemented by hard-label cross-entropy against the teacher's argmax,

$$\mathcal{L}_{\text{hard}} = \text{CE}\big(z^S, \arg\max z^T\big) \tag{5}$$

At the representation level, the student's 258-dimensional correlation cue is projected through a learned linear layer to the teacher's 512-dimensional space and matched via MSE,

$$\mathcal{L}_{\text{corr}} = \big\| P(c^S) - c^T \big\|^2 \tag{6}$$

where P is discarded after training. This alignment is critical because the student's dot-product correlation is structurally different from the teacher's learned pyramid, and action-level supervision alone is too sparse to recover the spatial reasoning.

The total loss combines these with weights $\alpha_{\text{KL}} = 1.0$, $\alpha_{\text{corr}} = 0.5$, $\alpha_{\text{hard}} = 0.5$, trained for 100,000 steps with Adam and cosine annealing from lr $= 10^{-3}$.

Quantization and Export

After distillation, fake quantization nodes (per-tensor symmetric INT8) are inserted and the model is fine-tuned for 10,000 additional steps at lr $= 10^{-4}$ with continued distillation loss.

The quantized student is split into two TFLite models for deployment. The TinyEncoder and TinyPolicy are exported separately because the cross-correlation between them is implemented in hand-written C on the ESP32-S3, avoiding the need for a custom TFLite operator. The TFLite flatbuffers are converted to C arrays and embedded directly in firmware flash. At inference time, two TFLite Micro interpreters are allocated from PSRAM (384 KB for the encoder, 32 KB for the policy). The goal image is encoded once at the start of each navigation episode and cached, so each step requires only one encoder pass on the current observation, the C correlation, and the policy MLP.

References

[1] Peter Anderson, Angel Chang, Devendra Singh Chaplot, Alexey Dosovitskiy, Saurabh Gupta, Vladlen Koltun, Jana Kosecka, Jitendra Malik, Roozbeh Mottaghi, Manolis Savva, and Amir R. Zamir. On evaluation of embodied navigation agents, 2018.

[2] Devendra Singh Chaplot, Ruslan Salakhutdinov, Abhinav Gupta, and Saurabh Gupta. Neural topological slam for visual navigation, 2020.

[3] Vishnu Sashank Dorbala and Dinesh Manocha. Deploying foundation models for embodied navigation, 2026.

[4] Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning, 2021.

[5] Utkarsh Grover, Ravi Ranjan, Mingyang Mao, Trung Tien Dong, Satvik Praveen, Zhenqi Wu, J. Morris Chang, Tinoosh Mohsenin, Yi Sheng, Agoritsa Polyzou, Eiman Kanjo, and Xiaomin Lin. Embodied foundation models at the edge: A survey of deployment constraints and mitigation strategies, 2026.

[6] Meera Hahn, Devendra Chaplot, Shubham Tulsiani, Mustafa Mukadam, James M. Rehg, and Abhinav Gupta. No rl, no simulation: Learning to navigate without navigating, 2021.

[7] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017.

[8] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning, 2021.

[9] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices, 2020.

[10] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. On-device training under 256kb memory, 2024.

[11] Xiaoming Liu, Borong Zhang, Qingbiao Li, and Steven Morad. 120 minutes and a laptop: Minimalist image-goal navigation via unsupervised exploration and offline rl, 2026.

[12] Arjun Majumdar, Gunjan Aggarwal, Bhavika Devnani, Judy Hoffman, and Dhruv Batra. Zson: Zero-shot object-goal navigation using multimodal goal embeddings, 2023.

[13] Gianluca Monaci, Philippe Weinzaepfel, and Christian Wolf. What does really matter in image goal navigation?, 2026.

[14] Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with hilbert representations, 2024.

[15] Zheng Qin, Le Wang, Yabing Wang, Sanping Zhou, Gang Hua, and Wei Tang. Rsrnav: Reasoning spatial relationship for image-goal navigation, 2025.

[16] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A platform for embodied ai research, 2019.

[17] Dhruv Shah, Ajay Sridhar, Arjun Bhorkar, Noriaki Hirose, and Sergey Levine. Gnm: A general navigation model to drive any robot, 2023.

[18] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. Vint: A foundation model for visual navigation, 2023.

[19] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. Nomad: Goal masked diffusion policies for navigation and exploration, 2023.

[20] Xinyu Sun, Peihao Chen, Jugang Fan, Thomas H. Li, Jian Chen, and Mingkui Tan. Fg-prompt: Fine-grained goal prompting for image-goal navigation, 2023.

[21] Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow, 2020.

[22] Karmesh Yadav, Arjun Majumdar, Ram Ramrakhya, Naoki Yokoyama, Alexei Baevski, Zsolt Kira, Oleksandr Maksymets, and Dhruv Batra. Ovrl-v2: A simple state-of-art baseline for imagenav and objectnav, 2023.

[23] Zichen Yan, Rui Huang, Lei He, Shao Guo, and Lin Zhao. Sign: Safety-aware image-goal navigation for autonomous drones via reinforcement learning. *IEEE Robotics and Automation Letters*, 11(2):1962–1969, February 2026.

[24] Yuke Zhu, Roozbeh Mottaghi, Eric Kolve, Joseph J. Lim, Abhinav Gupta, Li Fei-Fei, and Ali Farhadi. Target-driven visual navigation in indoor scenes using deep reinforcement learning, 2016.